Whitepaper · free for the SAP community

SAP Clean Core Whitepaper

From custom ABAP nobody understands to a reviewed, tested rebuild — on one chain of evidence you can check.

For the person who has to decide what happens to a custom ABAP program, Clean-Core.io is the free workspace that reads the code before any model does, draws the business process as BPMN with a line anchor on every element, or the reason it has none, shows SAP's clean core level for each SAP object it touches, or Unknown where SAP's published data holds no answer — and then carries the same evidence through the whole way: a target design, a transformed code draft and test scenarios, run in an isolated runner or in your own SAP system, all traceable to the lines they came from and sealed as signed runs. Other tools explain code, or rewrite it. Clean-Core.io does both on one chain of evidence — and says what it could not determine.

The code is a draft for review. Tests here run in an isolated runner against mocks, not in your S/4HANA system; an ABAP Unit result from your own system is recorded as imported or self-declared. A signature proves where a run came from and that it is unchanged — not that it is right.

EDITION V3.0.11 OCTOBER 12, 2026 FELIX FRENZEL · CLEAN-CORE.IO PUBLIC

Read online at clean-core.io/whitepaper · the demo project is in every free account

CONTENTS

01 What is Clean-Core.io?
02 How is the process reconstructed from ABAP?
03 One case, three views
04 Where each SAP object stands
05 The tools in your workspace
06 How do I know what is proven and what is not?
07 Verify it yourself
08 Does it replace SAP's own tools?
09 Your data stays yours
10 How do I start, and what does it cost?
11 Questions people ask first

01 · FOR DECISION MAKERS

What is Clean-Core.io?

Other tools explain code, or rewrite it. Clean-Core.io does both — on one chain of evidence.

THE SITUATION

A custom program nobody can explain any more

The documentation was never written or is long gone, the person who built it has left, and the code is the only document that still says what the program does. Before an S/4HANA move or a clean core programme, someone has to decide what happens to it: keep it, rebuild it, move to SAP standard, or retire it.

WHAT YOU GET

The process, the rules and SAP's view of every object

The process the program runs, drawn as BPMN in plain language. The business rules hard-coded in it, each with its line. SAP's clean core level for each SAP object it touches, or Unknown where SAP's data holds no answer. Then a target design, a code draft and tests on the same evidence — and a signed record of the analysis they rest on.

WHAT STAYS WITH YOU

The decision

Clean-Core.io prepares the decision; it does not take it. You confirm the rules, choose the target and decide per object. What a model suggested is marked as a proposal until someone confirms it, and what the code does not show is listed as not determined.

TOOLS THAT EXPLAIN CODE

Explain what a program does — in words, a summary or a diagram.

TOOLS THAT REWRITE CODE

Produce new code from the old program.

CLEAN-CORE.IO

Explains, rebuilds, tests — one chain of evidence.

Reads your code before any model does, so every finding points to a line. Says what it could not determine, and never passes an assumption off as a fact.

One chain of evidence

Five steps on the same evidence. Each one says where it stands, and each points back to the line of code it came from.

1

Process

The business process as BPMN, read from the code. Every element points to its line, or says why it has none — the plant 1000 rule of the demo to L87.

Reconstructed · line anchor

2

Design

A target design for the route the evidence points to, built on the run the server signed — a proposal until you record the target you accept.

Model proposal · run reference

3

Code draft

The transformed code, generated from the source, the analysis and the design; its plan names every finding at its line. A draft you review, not a finished product.

Model proposal · line anchor

4

Tests

Test scenarios for the generated code. On the CAP track they run in an isolated runner against mocks, and the server records what ran, on which code; on the RAP track you record the ABAP Unit result from your own SAP system.

Demonstrated · mock · test receipt

5

Handover

An audit pack the server signs over the run, with HMAC and Ed25519, that anyone can verify offline.

Proven · signature

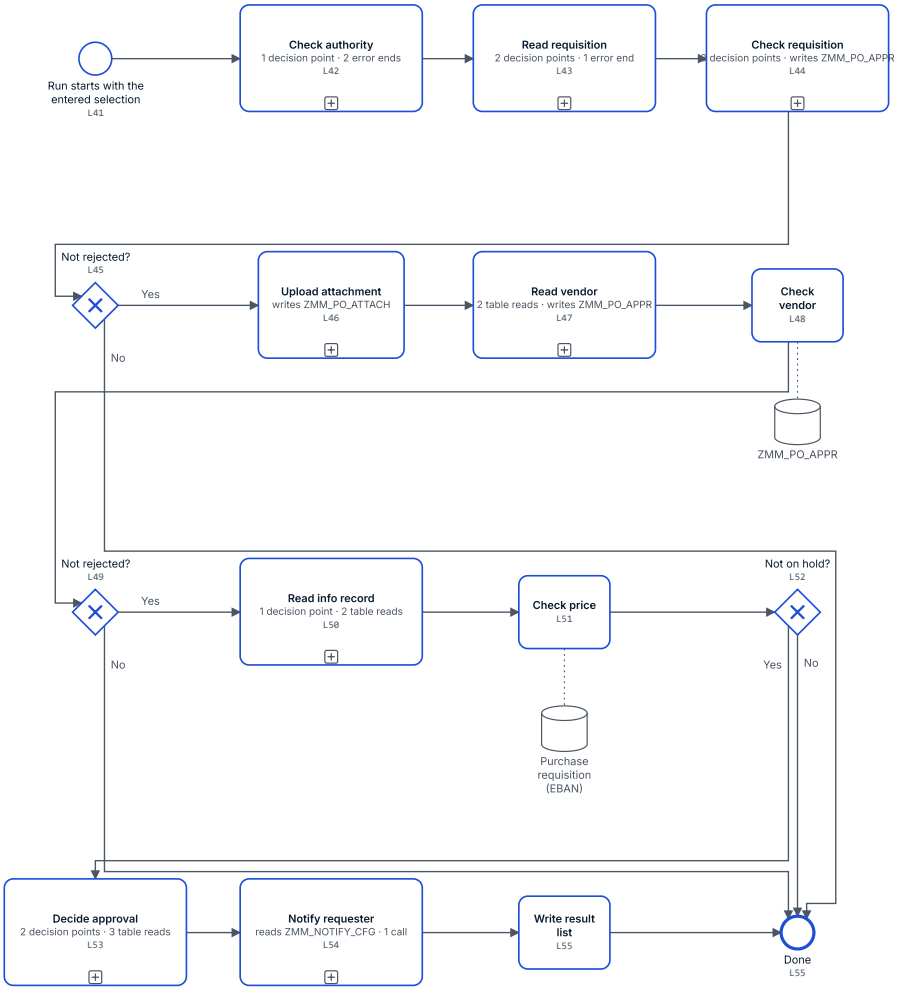
The code is a draft for review, not a finished product. On the CAP track the tests run in an isolated runner against mocks: they check the generated code against test scenarios, not that it runs in your S/4HANA system. On the RAP track nothing runs here: the ABAP Unit result comes from your own system, as an imported file or your own confirmation, never as proven. A signature proves where a run came from and that it is unchanged — not that it is right.

How is the process reconstructed from ABAP?

Clean-Core.io draws the process as BPMN from what the ABAP code does, puts a line anchor on every element, or the reason it has none, and names what the code cannot show instead of drawing it.

The process of Z_MM_PO_APPROVAL, reconstructed from the code

Demo project · fictitious code. The top level of the program as the BPMN 2.0 export writes it — collapsed sub-processes open into the next level. 84 of 84 elements of the whole program carry a line anchor.



Every element points to its lines

Start and end events, tasks, decision points and sub-processes each carry a line anchor. Decision points keep their condition from the code; proposed lanes are marked as proposals, never as your organisation.

Business rules come out of the code

Literals in conditions — tolerances, plants, vendor lists, date limits — become rule candidates with their anchor. You keep, change or drop each one, or mark it to clarify.

Unreached code is named, not drawn

Forms no entry point calls stay off the map and are listed underneath with their lines. Identical forms are grouped, technical helpers fold into their caller.

Leaves as a BPMN 2.0 XML file

Export the process as standard BPMN 2.0 XML; collapsed sub-processes stay real sub-processes. Import into SAP Signavio has not been verified yet. There is no connection to a Signavio workspace — only files.

Plain language first

Steps carry business names and decision points read as questions, worded from the code without a model. The names the code itself uses are one switch away — “Technical names” — for the developer.

Large processes stay readable

Levels instead of zoom: the map opens as an overview of phases. Open a phase in place, follow the path at the top back up, or read the same level as a list of steps — with the mouse or the keyboard.

Edit it in a BPMN editor

Every save is a new revision. The reconstruction from the code stays revision 1 and is never overwritten, and any revision can be compared with that as-is process: added, changed, removed.

Bring a model back in

A BPMN 2.0 XML file can be imported into the editor. It becomes a proposal for the next revision: elements it recognises keep their line anchor, elements added outside are marked as having none.

What the code touches is drawn too

Tables a step reads or writes appear as data stores beside it, and calls to another system as a collapsed pool — so a reader sees where the data comes from and where it goes.

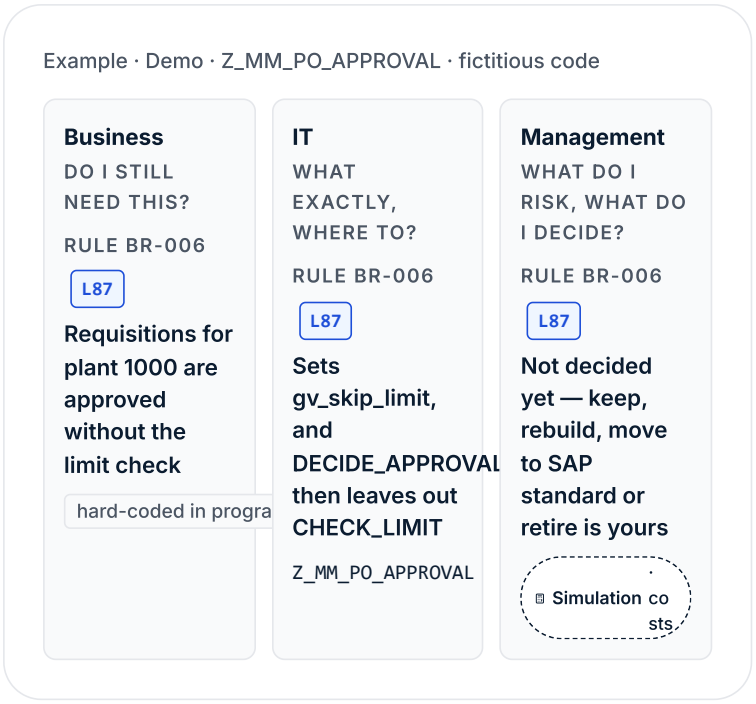
Decide per rule

For every business rule the code holds: keep it, change it deliberately, drop it, or mark it to clarify. Each answer is recorded with the account that gave it.

03 · THREE VIEWS

One case, three views

Business, IT and Management look at the same facts. Each view answers its own question — Do I still need this? What exactly, where to? What do I risk, what do I decide? — and none of them changes a result.



A view orders what you see. It is never stored with a project, a run, a signature or an audit pack, and it changes no result. You confirm as the signed-in account — a self-declaration, not an organisational mandate.

Business

Do I still need this, and what changes for me?

Opens with one next step, then the process as numbered steps in plain words with their lines, what it decides and what it changes, and the business rules — the hard-coded ones too — unanswered first.

IT

What exactly, where to, and is it right?

Opens with the findings at their line, the successor named and who names it, and the chain from requirement to anchor, finding and target draft.

Management

What do I risk, what do I decide?

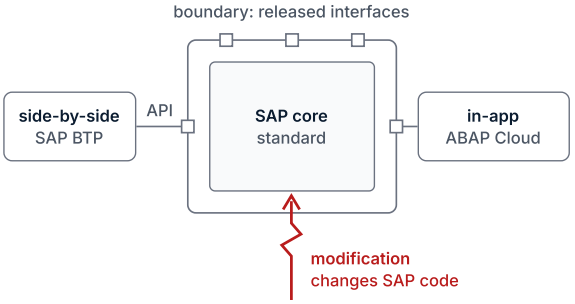
Opens with the decision — keep, rebuild, move to SAP standard or retire — what it rests on, the open conditions in plain words and fit to standard as one figure; costs only as a simulation.

Where each SAP object stands

Clean core keeps SAP S/4HANA standard. Clean core extensibility means extensions use only released, upgrade-stable interfaces — in-app with ABAP Cloud or side-by-side on SAP BTP, part of the SAP Business AI Platform.

What clean core means

Keep the SAP core standard: extensions use only released, upgrade-stable interfaces — in-app with ABAP Cloud or side-by-side on SAP BTP.



The four levels

The clean core level of each SAP object the code touches — or Unknown where SAP's data holds no answer — derived from the states SAP publishes, with a real SAP object on each level.

A **Released SAP APIs and extension points**

`I_SALESDOCUMENT` Imported · SAP release data Released CDS view.

B **Classic SAP APIs, following SAP recommendations**

`BAPI_PO_CREATE1` Imported · SAP release data Classic BAPI SAP still recommends for classic ABAP.

C **Internal SAP APIs — conditionally clean**

`REUSE_ALV_GRID_DISPLAY` Imported · SAP release data An SAP object neither repository file lists — internal by default.

D **Not recommended — modifications and retired technology**

`VBAK` Imported · SAP release data Sales document table, not to be released.

Levels follow SAP's clean core level concept. The level shown for an object is our reading of SAP's published data — an orientation, never part of a signed audit pack. Confirm with ABAP Test Cockpit.

How levels are assigned (clean-core.io/method/levels)Object classification A–D (clean-core.io/sap-clean-core-object-classification)

Target profile: Public Edition or Private Edition

Each project names the S/4HANA edition it moves to. Every SAP object then lands in one of four buckets — the first rule that applies wins, and every assignment shows its evidence.

Retire

The rule it serves is confirmed “Drop”, or a usage import shows no executions over at least 13 months — then it is a candidate, an open check, not a verdict.

The object goes away. Nothing has to be built for it.

No catalogued path

Needed, below the target platform’s bar, and SAP’s Cloudification Repository names no released API and no successor for it.

Nobody has named a way yet. SAP's repository names no released API and no successor for these objects, or does not list them at all, so the next step is a question someone has to answer — not work this project can plan, and not a fault in this code.

Rebuild

Needed, below the bar, and a successor or an extension path is named — and every modification or own write to an SAP table.

The way is known and the work sits with this project: SAP names a successor or an extension path to move to, or the object is the project's own work — its own code, a modification or a direct write.

Keep

Needed and permitted on the target platform: Public Edition level A only, Private Edition level A or B.

The object stays as it is on the chosen platform. Nothing has to be built for it.

The same level B object is Keep in Private Edition and Rebuild or No catalogued path in Public Edition. Change the target platform and the page says what moved.

The Clean Core Score

One figure from 5 to 100 for how far the analysed code is decoupled from the SAP standard core — higher is better. A grade, not a compliance percentage — and not an SAP measure: SAP publishes no metric of that name, and it is not SAP's Technical Debt Score, which runs the other way.

`91–100`	**Close to clean core** Nothing heavy in what was read — at most lighter constructs such as reads of SAP tables, classic screens or remote calls, and in Private Edition writes to the program's own Z/Y tables.
`81–90`	**Some rework** No modification of SAP code and no direct write to an SAP standard table — other constructs need work.
`60–80`	**Significant rework** One construct of real weight, or several lighter ones together, has to be replaced or moved.
`5–59`	**Far from clean core** Several kinds of construct, or parts the engine could not assess, stand in the way of the clean core rules.

Clean-Core.io's bands, derived from the deductions.

What the Clean Core Score measures (clean-core.io/clean-core-score)

Where the evidence comes from

1

Your ABAP source Reconstructed

Every statement keeps its program, include and line — that is what a line anchor points to. Includes that were not uploaded are named as not determined.

2

Deterministic engine Reconstructed

Parses the code and finds the constructs, rules and SAP objects without a language model. The same file gives the same result.

3

SAP’s published data Imported

The Cloudification Repository and SAP’s object classification, 33,877 objects, synced 2026-10-01.

4

Your imports, optional Imported

ATC results and usage data you upload. They are marked as imported, never as proven.

5

A language model Model proposal

Business names and drafts come last and are marked as a model proposal until someone confirms them.

05 · THE SEVEN STAGES

The tools in your workspace

Every project opens as one workspace: one next step and the process in numbered steps on top, the three views, and seven tools behind them. Each tool produces something you can read and check before the next — and a person signs the result, not the tool.

1 Analyze

A deterministic engine reads the ABAP before any model does: every finding at its line, a map of where in the program they sit, and the Clean Core Score on a scale that says what the number means and what moved it.

The result is recorded as a signed run, the extensibility route follows from fixed rules, and every later stage builds on it.

Proven — Done by the deterministic engine

2 Design

The target architecture as a canvas, drawn from the run: what stays inside SAP S/4HANA behind the clean core boundary, which released API replaces which lines, what has no released successor, and on the side-by-side track what runs on SAP BTP, part of the SAP Business AI Platform.

A model writes the design document beside it when the stage opens; you record which target you accept — a self-declaration, not a mandate.

Reconstructed · Model proposal · Confirmed

Drafted by a model — a person decides

3 Transformation

A model generates the target code from the source, the analysis and the design: ABAP Cloud artefacts on the RAP track, a Node.js project on the CAP track. Beside it, the engine's plan shows where every finding goes.

Nothing compiles or tests it in this stage; it is a draft you review.

Model proposal — Drafted by a model — a person decides

4 Documentation

When the stage opens, the analysed source is written up as a process description a successor can read — what it covers, how a run starts, the process with its map and who acts at each step, decision points and business rules, exceptions, what it changes, systems and data, and open questions — every step with its line, details and evidence as an appendix.

It exports to Confluence, Markdown and Word; a business layer with SOPs and a RACI matrix is a model draft, written only when you ask for it.

Reconstructed · Model proposal

Done by the deterministic engine

5 Testing

A model writes test scenarios for the generated code; on the CAP track they run against mocks in an isolated test runner, never on your system. The engine lists what a tester has to check by hand, with the lines.

On the RAP track the scenarios are an ABAP Unit class that runs only in your own system: you upload its result file or confirm the result yourself, marked as imported or self-declared, never as proven. A pass against mocks never means the code works in SAP.

Model proposal · Demonstrated · mock · Imported · Confirmed — Drafted by a model — a person decides

6 Economics

A demonstration model compares doing nothing, keeping the code and moving to SAP standard on cost figures you enter and assumed coefficients, and lists every input still open.

It is not a business case, and it shows no forecast until your own figures are in.

Simulation — Done by the deterministic engine

7 Delivery

Delivery shows the chain from requirement to decision, receipt and artefact, what a handover still needs, and the audit pack the server signs over the signed run — blocked while anything was built for a previous source.

A pack can be checked on the Verify Pack page; whether to deploy stays an architect's decision.

Proven — Done by the deterministic engine

Delivery lets you export the generated package — the files in standard abapGit layout and the generated tests — to compile, activate and test in your own system. Economics shows costs only as a simulation on figures you enter.

Familiar to SAP users

The workspace and its tools follow SAP Fiori design patterns — a list of your projects, an object page per project, one clear next step. It is an independent design: not a Fiori app, and not affiliated with or endorsed by SAP.

Share by invitation

Only the account that created a project, and anyone that account invites, can open it. An invitation is bound to one confirmed e-mail address and gives read access including the source code until it is withdrawn, which can happen at any time. Its link expires if nobody accepts it. There are no public links.

A print sheet per project

One printable sheet: the process as numbered steps, the business rules with the answer on record, every SAP object the code names with its clean core level A–D or Unknown, and everything that was not determined. The level prints as orientation — it is not part of the signed run.

06 · HONEST BY DESIGN

How do I know what is proven and what is not?

Every statement carries its source as a word, an icon and a shape. Nothing simulated passes as proven: missing stays missing, simulated stays simulated, reconstructed stays reconstructed.

Where a statement comes from

One fixed list. The shape says how firm a statement is.

FILLED settled	
Proven	Backed by the engine, a signature or a real run.
Confirmed	Confirmed by the signed-in account — a self-declaration, not a mandate.
Stale	No longer current — recompute.
OUTLINE derived or imported	
Reconstructed	Derived from the code, not confirmed by anyone.
Imported	Taken from a file — ATC, BPMN, usage, an SAP catalogue.
Not determined	Could not be determined — the reason is given.
DASHED provisional	
Model proposal	Proposed by the language model, unchecked.
Simulation	Computed on assumptions you entered.
Demonstrated · mock	Shown against mocks, not against a real system.

Signed, and what a signature means

What the code does not show is listed as not determined, with its reason and line — never guessed.

Signed runs. Every completed analysis is sealed as a signed, unchangeable run. Every later tool builds on that run, and says when the source has changed since.

An audit pack anyone can check. The server signs the audit pack over the run with HMAC and Ed25519. The Ed25519 signature checks against a published public key — on the Verify Pack page, or offline with the open-source verifier, without an account and without trusting us.

What a signature does not say. It proves where a run came from and that it is unchanged — not that the result is right. The clean core level A–D is never part of the signed pack, and a demo run is never signed.

Verify an audit pack (clean-core.io/verify-pack)**Audit evidence** (clean-core.io/features/audit-evidence)

07 · ONE REPRODUCIBLE RUN

Verify it yourself

Run the published 907-line example and you get the same result: the engine is deterministic, the source file ships with the product, and every completed analysis is sealed as a signed run you can verify.

The reference run

`ZLEGACY_ORDER_FULFILLMENT_AUDIT_1000LOC.abap`, analysed by the same engine the product uses, computed from the file when the page is built.

907 lines of code

42 findings

45 Clean Core Score

Score 45 of 100 · far from clean core (5–59).

23 **the tool settles**
Points at a released SAP successor — SAP's own where it names one, the curated mapping otherwise. The finding says which. You review it; you do not have to find it.

15 **your decision**
Transformable, but somebody has to weigh business intent against the target design. The tool shows the evidence and stops.

4 **stays hand work**
Out of reach for any generator. Flagged and isolated rather than guessed at, so nothing false reaches your draft.

Of the 11 ABAP construct classes the engine tracks, 3 are fully covered, 6 require sign-off and 2 are not supported — published per class, before you upload anything. One synthetic reference program is not a codebase, and your ratio will differ.

Open the reference run (clean-core.io/reference-analysis)

SAP objects this run touched

With the successor from SAP's own release data. Where the engine uses a curated field-level mapping instead, the finding says so.

KNA1	→	I_CUSTOMER
KNB1	→	I_CUSTOMERCOMPANY
MARA	→	I_PRODUCT
MARD	→	I_PRODUCTSTORAGELOCATIONBASIC
VBAK	→	I_SALESDOCUMENT
VBAP	→	I_SALESDOCUMENTITEM

One finding lands on the business: Credit Management Custom Logic (L401). The engine's recommendation, quoted unedited: "Evaluate if SAP FSCM / Advanced Credit Management (F1007) covers this use case. If standard coverage is insufficient, implement remaining gap as a Side-by-Side microservice on SAP BTP."

1 · Import the draft
Import the generated abapGit package into Eclipse ADT.

2 · Compile and test
Compile the code, run the ABAP Unit tests in your own sandbox and record the result in Testing.

3 · Check the seal
Verify the signed audit pack — **verify a pack** (clean-core.io/verify-pack).

08 · NEXT TO YOUR SAP TOOLS

Does it replace SAP's own tools?

No. ABAP Test Cockpit stays the authoritative check, ADT stays where code is built and tested, and SAP's agents fix code inside your system; Clean-Core.io prepares the evidence and the decision around them.

SAP TOOL	WHAT IT IS FOR	HOW CLEAN-CORE.IO RELATES
ABAP Test Cockpit (ATC)	The authoritative check for clean core violations — keep using it.	Reads the same SAP Cloudification Repository (33,877 objects, classified) and shows its reading as level A–D. Import ATC results to compare them with the engine.
ABAP Development Tools (ADT)	Where ABAP is developed, compiled and unit-tested; ABAP Unit and the CDS Test Double Framework are on board.	Generated drafts arrive as an abapGit package that you import, compile and test there.
Joule for Developers and the Custom Code Migration Agent	SAP’s AI for ABAP developers, inside your system: it explains code and fixes what ATC finds, under SAP’s licence.	They tell developers what to fix. Clean-Core.io shows the business what the code does and carries the same evidence through a design, a code draft and tests to a decision; what stays goes to them. ATC stays the authority.
SAP Signavio	Process modelling, under its own licence.	The process leaves as a BPMN 2.0 XML file. Import into SAP Signavio has not been verified yet; there is no connection to a Signavio workspace.
SAP Cloud ALM	Project and application lifecycle, under SAP’s licence.	No adapter. The handover package is a set of files you take along.

What it deliberately does not do

- × Decide for you. The rules, the target and the decision per object stay with the people who own them.
- × Perform or guarantee an automated migration, or deliver production-ready code without review.
- × Replace SAP’s own tools: ABAP Test Cockpit stays the authority, ADT stays where code is built and tested.
- × Transform SAP GUI dynpro screens, core modifications, native SQL, dynamic call routing or kernel internals — these are flagged and handed back, never guessed at.

- × Promise a time saving or a business case. Costs appear only as a simulation on figures you enter.
- × Claim any affiliation with or endorsement by SAP SE.
- × Connect to SAP Signavio, SAP Cloud ALM or your S/4HANA system to change anything. What leaves is files.

09 · YOUR CODE AND YOUR TRUST

Your data stays yours

Your code is stored in the EU, reaches the Google Gemini API only through our server, and is never sold or commercially used. Each line below links to the document that says so.

What we do so you can trust us

- Stored in the EU — Google Cloud, Belgium (europe-west1).
  Privacy Policy §4
- Only the account that created this project can open it, and anyone that account chooses to invite; no one else has standing access.
  Privacy Policy §8
- Model calls go through our server; keys never reach the browser. Your own key is stored encrypted.
  Terms §5
- Every completed analysis is sealed as a signed, unchangeable run.
  Security model §14.1
- No analytics, advertising or tracking cookies.
  Privacy Policy §7
- Delete your account and its projects at any time; backup copies age out within 30 days.
  Privacy Policy §5
- With our community key, Google does not use your code to train its models — the paid Gemini API terms apply. With your own key, your Google account's terms apply.
  Privacy Policy §3
- Our security model is public.
  SECURITY.md

A free community project

Clean-Core.io is a free community project: there is no paid tier, we accept no payment, and we do not sell, rent or commercially use your code.

Clean-Core.io is independent and not affiliated with or endorsed by SAP SE. Generated output is a draft that you review before any productive use. The code of this product is public: github.com/sonnyfrenzel-rgb/clean-core.io.

10 · START · FREE

How do I start, and what does it cost?

Nothing — Clean-Core.io is free. Each of the 8 examples runs once at no cost, your own code uses one of five free analysis runs, and after that you continue with your own Gemini API key.

1 **Create a free account**

Sign up with Google or with e-mail and password. No payment, no card.

2 **Open the demo, an example or your code**

The engine reads the source first. The process, its rules and what could not be determined appear with line anchors.

3 **Answer and decide**

Answer the rules — keep, change, drop or clarify — record the decision for the program and hand over. Every completed analysis is sealed as a signed, unchangeable run.

The demo project and its tour

Every account has the same demo project: the engine's reading of the example Z_MM_PO_APPROVAL on every stage — fictitious code, a guided tour, and nothing you do there is saved or counted. A demo run is never signed.

1. First insights into your process
2. The process map and its source
3. Levels of a large process
4. What we could not determine
5. Decide on rules
6. Standard fit
7. The IT chain
8. The Management view
9. Four buckets
10. Costs are a simulation
11. The decision
12. Handover

Read on

How it works, and its limits (clean-core.io/how-it-works)**Clean core, explained without the jargon** (clean-core.io/clean-core-explained)**Object classification A–D** (clean-core.io/sap-clean-core-object-classification)**The SAP object catalog** (clean-core.io/catalog)**Facts and figures** (clean-core.io/facts)**The security model on GitHub**

FF

Felix Frenzel

Founder & community builder, Clean-Core.io · Bamberg, Germany

11 · FAQ

Questions people ask first

What is SAP clean core?

Clean core keeps the SAP S/4HANA standard unmodified: extensions use only released, upgrade-stable interfaces — in-app with ABAP Cloud or side-by-side on SAP BTP, part of the SAP Business AI Platform. SAP's clean core level concept grades what an extension uses from A (released APIs and extension points) to D (not recommended: modifications, implicit enhancements, writes to SAP tables).

Clean core, explained without the jargon (clean-core.io/clean-core-explained)

What does Clean-Core.io do with my ABAP?

A deterministic engine reads the program before any language model does. It reconstructs the business process with a line anchor on every element, or the reason it has none, lists the business rules hard-coded in the program, shows the clean core level of each SAP object the code uses, or Unknown where SAP's published data holds no answer, and names what it could not determine. The same evidence then carries a target design, a transformed code draft and test scenarios — on the CAP track run against mocks in an isolated runner, on the RAP track an ABAP Unit class for your own system, whose result you record from that system — and every completed analysis is sealed as a signed run. The code is a draft you review; a run against mocks checks it against test scenarios, not in your S/4HANA system.

How it works, and its limits (clean-core.io/how-it-works)

What is Clean-Core.io as of v3.0.x?

As of v3.0.11 (released 2026-10-12), Clean-Core.io is a free, independent web workspace for deciding what happens to one custom ABAP program. A deterministic engine reads the code before any language model does and reconstructs the business process as BPMN, with a line anchor on every element or the reason it has none, including the business rules hard-coded in the program. Each SAP object the code uses gets a Level A–D under a versioned rule, read from SAP's published Cloudification Repository and object classification, or Unknown where neither holds an answer. On that evidence the Management view asks for one decision — keep, rebuild, move to SAP standard or retire — and Design, Transformation and Testing carry the same evidence to a target design, a code draft and test scenarios, each marked as a Model proposal for a person to review. Every completed analysis is stored as an immutable run signed by the server with HMAC; the audit pack exported from it is signed with HMAC and Ed25519, and its Ed25519 signature can be verified offline. What the engine could not determine is listed as not determined, not guessed.

Facts, versions and sources (clean-core.io/facts)

What changed in Clean-Core.io 3.0?

Clean-Core.io 3.0, released 2026-10-03, gives every project one workspace, and the seven stages are no longer the product: they are tools inside it. A project opens on the business process reconstructed from its ABAP code as BPMN, shown in three views of the same facts — Business, IT and Management. A view orders what is shown; it changes no result and is never stored on a run or a signature. The Management view leads with the decision — keep, rebuild, move to SAP standard or retire. Also new in 3.0: a BPMN editor in which every save is an immutable revision and the reconstruction stays revision 1, with BPMN 2.0 XML files in and out and no connection to a Signavio workspace; documentation written from the engine's reading of the code, no longer by a model reading only the start of it; and a demo project with a guided tour in every account. Sign-up and accounts did not change. The releases since then, up to v3.0.11 (2026-10-12), refine 3.0; the release notes are in CHANGELOG.md of the public repository, github.com/sonnyfrenzel-rgb/clean-core.io.

How does Clean-Core.io turn ABAP code into a BPMN process?

Clean-Core.io's deterministic engine reads the ABAP program and reconstructs the business process it runs as a BPMN 2.0 diagram: every task, gateway and event points to the ABAP line it came from, or says why it has none. Business names that a language model suggests are marked as a model proposal. The process can be edited in the built-in BPMN editor; every save is a new revision, and the reconstruction itself is never overwritten.

Process blueprints (clean-core.io/features/process-blueprints)

Is my code used to train models?

With our community key, Google does not use your code to train its models — the paid Gemini API terms apply. With your own key, your Google account's terms apply.

Privacy Policy §3 (clean-core.io/datenschutz#source-code)

What does it cost?

Nothing. Clean-Core.io is a free community project with no paid tier, and we accept no payment. An account has five analysis runs; each starter example is free the first time you run it. After that you continue with your own Gemini API key, which Google bills under your own agreement with Google.

Is the generated code ready to go live?

No. The transformed RAP or CAP code from Clean-Core.io is a draft for a person to review. On the CAP track its test scenarios run in an isolated runner against mocks, so they do not show that the code runs in a real SAP S/4HANA system; on the RAP track they are an ABAP Unit class that runs only in your own system, and nothing is run here — you record its result as an imported file or your own confirmation, never as proven. A signed run proves where a result came from and that it has not changed since — not that it is correct. What the engine could not determine is listed as not determined rather than guessed.

How it works, and its limits (clean-core.io/how-it-works)

Does it replace ABAP Test Cockpit?

No. ABAP Test Cockpit stays the check to rely on. The level Clean-Core.io shows is its reading of SAP's published data — an orientation, never part of a signed audit pack. Confirm with ABAP Test Cockpit, and import your ATC results to compare them with the engine.

Does it replace Joule for Developers or SAP's Custom Code Migration Agent?

No. SAP's agents work inside your system and tell developers what to fix. Clean-Core.io shows the business what the code does, then carries the same evidence through a design, a code draft and tests to a decision — including whether the program is still needed at all. What stays goes to the developers and their tools, and ABAP Test Cockpit stays the authority.

Does it work with SAP Signavio?

Clean-Core.io exchanges standard BPMN 2.0 XML files: it exports the reconstructed process, and a BPMN 2.0 XML file can be imported back into its process editor as a new revision. Import into SAP Signavio has not been verified yet, so we do not claim it, and there is no connection to a Signavio workspace.

What are the Business, IT and Management views?

Three ways to read the same Clean-Core.io project. The Business view answers "Do I still need this, and what changes for me?", the IT view "What exactly, where to, and is it right?", and the Management view "What do I risk, what do I decide?". A view only orders what is shown; it changes no result, and it is never stored on a run or a signature.

What is the Clean Core Score?

The Clean Core Score is a 5–100 measure, published by Clean-Core.io, of how far the analysed custom ABAP is decoupled from the SAP standard core; higher is better. It falls into four bands: Far from clean core (5–59), Significant rework (60–80), Some rework (81–90), Close to clean core (91–100). SAP publishes no metric of that name, and it is not SAP's Technical Debt Score, which runs the other way.

What the Clean Core Score measures (clean-core.io/clean-core-score)

What is the SAP Cloudification Repository viewer?

The SAP object catalog on Clean-Core.io shows SAP's published Cloudification Repository and object classification: for each SAP object its release state, the clean core level Clean-Core.io derives from it and, where SAP names one, its successor — synced from SAP's public repository. It needs no account.

Open the SAP object catalog (clean-core.io/catalog)

Who can see my projects?

Only the account that created a project, and anyone that account invites. An invitation is bound to one confirmed e-mail address and gives read access including the source code until it is withdrawn, which can happen at any time. Its link expires if nobody accepts it. There are no public links.

Where is my data stored?

Stored in the EU — Google Cloud, Belgium (europe-west1). No analytics, advertising or tracking cookies. Delete your account and its projects at any time; backup copies age out within 30 days.

Trust and privacy (clean-core.io/trust)

Can I try the demo without an account?

No. The demo project lives in your workspace, so it needs a free account. Inside it nothing you do is saved, and nothing counts against your analysis runs.

How does clean core reduce S/4HANA upgrade risk?

Custom code that reads or modifies the SAP standard directly is what makes an upgrade expensive: a modification has to be adjusted in SPAU before the upgrade can proceed, native SQL bypasses the database abstraction, and every direct table read relies on a structure SAP never promised to keep. Clean core replaces those with released APIs. Clean-Core.io names them in your own ABAP, object by object against SAP's published Cloudification Repository, and flags what a generator cannot reach instead of transforming it into something plausible and wrong.

Is Clean-Core.io an SAP product?

No. It is an independent community project, not affiliated with or endorsed by SAP SE. It follows SAP's clean core level concept and reads SAP's published Cloudification Repository.

Read the code before you decide.

Generated output is a draft that you review, test and approve before any productive use (**Terms of Service**). The platform is provided free of charge and without warranty.

SAP, S/4HANA, ABAP, SAP BTP, part of the SAP Business AI Platform and related names are trademarks or registered trademarks of SAP SE in Germany and other countries. Clean-Core.io is an independent, community-built tool and is **not affiliated with, sponsored, certified, or endorsed by SAP SE**. All SAP marks are used nominatively for identification only.

© 2026 Clean-Core.io · Whitepaper, edition v3.0.11 · October 12, 2026 · clean-core.io/whitepaper